

SYNOPSIS

1. Project Title

SoundHarbour — MERN Stack Real-Time Social Music Streaming Web Application

2. Statement of the Problem

With the rapid growth of digital media, users expect music platforms that are not only functional for playback and playlist management but also socially connected. Mainstream platforms such as Spotify, Apple Music, and YouTube Music treat music consumption as a largely passive, isolated activity. While they offer public playlists and activity feeds, none provides a native mechanism for sharing a song directly inside a conversation with a specific friend in a way that is instantly playable without leaving the chat window.

Additionally, existing open-source alternatives rarely combine all three pillars, audio streaming, social networking, and real-time messaging, in a single integrated application. There is a clear need for a full-stack web application that:

Allows users to stream self-hosted audio without dependence on third-party licensing APIs.

Supports a social graph (follow / friend system) with a real-time presence indicator.

Embeds shareable, playable song and playlist cards directly inside direct-message conversations.

Provides personalised listening statistics and a privacy-controlled user profile.

SoundHarbour addresses this gap by building a complete, self-contained MERN stack platform that unifies music streaming, social networking, and real-time chat in a single responsive web interface.

3. Why This Topic Was Chosen

This project was chosen because:

Real-world relevance: Music streaming combined with social features represents the next evolution of consumer media apps, reflecting industry trends in products like Spotify's friend activity, Discord's music bots, and Apple Music's SharePlay.

Technical breadth: The project requires mastery of the complete MERN stack, React 18 front-end with context API and lazy-loaded routing, Express/Node.js REST API, MongoDB with aggregation pipelines, JWT authentication, Socket.io real-time communication, and file streaming, making it an ideal capstone to demonstrate full-stack competence.

No third-party API dependency: Unlike projects relying on the Spotify Web API (which requires OAuth approval and has preview-only limitations), SoundHarbour streams audio files hosted by the application itself, making it fully functional without external API keys or licensing agreements.

Social engineering challenge: Implementing a bi-directional friend request workflow, online presence broadcasting, real-time message delivery with read receipts, and typing indicators introduces distributed systems concepts that go beyond a standard CRUD application.

Extensibility: The platform is architected to accommodate future additions such as group chats, algorithmic recommendations, CDN delivery, or mobile native apps.

4. Objective and Scope of the Project

Objectives-

Build a secure user authentication system with registration, JWT-based login, role management (admin / listener), and session persistence.

Implement on-demand audio streaming via HTTP range requests so songs begin playing immediately without full download.

Allow users to create, manage, and share personal playlists; admin users can upload and delete songs from the platform library.

Design a social graph supporting one-way following and two-way friendship with a complete request workflow (send → accept / reject / cancel).

Build a real-time direct-messaging system with three message types: plain text, playable song cards, and shareable playlist cards.

Track per-user listening history and surface a personalised profile dashboard showing most-played songs, total plays, recent activity, and playlist count.

Provide a People discovery page with live name search and inline social actions.

Implement public/private profile visibility toggle so users control who can view their listening data.

Scope-

- Audio streaming (MP3/WAV)
- User auth with JWT
- Admin song upload/management
- Playlist CRUD
- Follow / Friend / Block graph
- Real-time DM chat
- In-chat playable song/playlist cards
- Share songs & playlists to friends``
- Online presence indicators
- Listening history & profile stats
- Public / private profile toggle
- Mobile-responsive dark-theme UI

5. Methodology

The project follows a feature-driven iterative development approach. Each sprint delivers a complete vertical slice, from database schema to REST endpoint to UI component, ensuring a working demo at every iteration boundary and surfacing integration bugs early.

Requirement Analysis

Identify user needs: authentication, browsing and streaming songs, creating playlists, managing a social graph, real-time messaging, and personalised statistics.

Determine tech stack requirements for a real-time, responsive, self-hosted web application.

System Design

Front-End: React 18 with Vite, React Router v6 (lazy-loaded route splitting), Context API for global state (SongContext, QueueContext, SocketContext, SidebarContext, FetchContext), and Tailwind CSS with CSS custom properties for a unified dark design system.

Back-End: Node.js + Express.js REST API at /api/v1/; Socket.io layered on the same HTTP server, authenticated via JWT in the WebSocket handshake; Multer for file uploads; HTTP range-request streaming for efficient on-demand audio delivery.

Database: MongoDB, collections: users, songs, playlists, follows, friendRequests, conversations, messages, playHistory. Aggregation pipelines compute per-user top-songs charts.

Implementation

Create UI components and pages using React and Tailwind CSS with a consistent dark theme.

Develop REST API modules: auth, song, playlist, user, follow, chat, each with its own controller, route file, and JWT/admin middleware guard.

Implement JWT-based authentication; store token in localStorage; decode on every protected request.

Build the real-time chat system: Socket.io rooms scoped per conversation; message types: text, song card (playable), playlist card (viewable).

Implement the ShareModal component on SongCard and PlaylistCard so users can send any song or playlist directly into a friend's chat with one click.

Track listening history on every song play; aggregate into most-played charts on the Profile page.

Implement online presence: server-side Map<userId, Set<socketId>>; broadcast presence events and full online-user list on socket connect.

Real-Time Chat Data Flow

- i. User A clicks Share on a SongCard → ShareModal loads friends list
- ii. User A selects Friend B and clicks Send
- iii. Client calls POST /api/v1/chat/with/B → server creates conversation if new
- iv. Client emits socket.emit('message', { type:'song', song:{...}, conversationId })
- v. Server saves message to MongoDB, broadcasts to conversation room via io.to(convId).emit('message', ...)
- vi. Friend B's Chat page receives the socket event → playable SongBubble appears in real time
- vii. Friend B taps the SongBubble → audio.src is set → song plays without leaving the chat
- viii. Integration and Testing
- ix. Manual functional testing of all user flows: register → login → play song → create playlist → share with friend → receive in chat → play from chat → view profile.
- x. API integration testing via Postman collections covering all REST endpoints with expected status codes and response shapes.
- xi. Real-time testing in two browser tabs logged in as different users to validate Socket.io presence, message delivery, and typing indicators.
- xii. Responsive design validation across mobile (375 px), tablet (768 px), and desktop (1440 px) using Chrome DevTools.
- xiii. Error-case testing: invalid route IDs, expired tokens, private profile access, non-friend chat attempts.

6. Hardware and Software to Be Used

Hardware-

Any standard development machine capable of running Node.js and a modern browser is sufficient: PC or Laptop with at least a dual-core processor and 4 GB RAM (8 GB recommended).

Stable broadband internet connection for npm package installation and MongoDB Atlas access.

Storage: at least 10 GB free for project files, dependencies, and uploaded audio files.

Software-

Category	Tools & Technologies
Operating System	Windows 10/11 / macOS / Linux
Front-End	React 18, Vite, React Router v6, Tailwind CSS, Context API
Back-End	Node.js v18 LTS, Express.js v4
Real-Time	Socket.io v4 (server + client)
Database	MongoDB (Atlas or local), Native MongoDB Driver
Authentication	JSON Web Tokens (JWT), bcryptjs
HTTP Client	Axios
File Upload	Multer (multipart form data)
Development Tools	VS Code, Postman, Git / GitHub, React DevTools
Build Tool	Vite (frontend bundler and dev server)

7. Testing Technologies Used

To ensure reliability and correctness across all features, the following testing approaches are used:
Manual Functional Testing: Every user-facing flow is tested end-to-end, registration, login, song playback, playlist operations, profile visibility toggle, friend request lifecycle, and in-chat media sharing.

API Integration Testing (Postman): Pre-written Postman collections test every REST endpoint with valid, invalid, and boundary-case request bodies; response status codes and JSON shapes are asserted.

Real-Time Socket Testing: Two browser sessions (different users) are run simultaneously to verify presence events (online/offline status), message delivery, read receipts, and typing indicators fire correctly.

Audio Player Testing: Seek-drag, play/pause, queue, skip-next, and volume controls are verified; the isSeeking ref pattern is validated to confirm timeupdate does not conflict with active drag.

Error Boundary Testing: Expired JWT, unauthorised playlist access, private profile URL, and non-friend chat requests are tested to confirm correct 401/403/404 responses and graceful UI error screens.

Cross-Browser Testing: Core functionality verified in Chrome, Firefox, and Edge; Web Audio API and MediaSource compatibility confirmed.

Responsive Design Testing: Chrome DevTools device emulation at 375 px (mobile), 768 px (tablet), and 1440 px (desktop) breakpoints ensures the layout adapts correctly on all screen sizes.

8. Project Contribution / Outcome

SoundHarbour delivers the following concrete contributions:

Functional full-stack application: A production-quality web platform combining music streaming, social networking, and real-time messaging in a single cohesive product.

Playable media cards in chat: The defining innovation, song and playlist bubbles inside DM conversations that play audio instantly without navigating away from the chat window. This pattern is not available in any mainstream messaging application today.

Self-hosted streaming: Unlike projects depending on the Spotify API, SoundHarbour streams its own uploaded audio via HTTP range requests, demonstrating real file-I/O and streaming engineering, not just API consumption.

Complete social graph: A fully implemented follow/friend system with request lifecycle, online presence, and People discovery with live search, demonstrating graph-style data modelling in MongoDB.

Privacy-first design: Per-user public/private profile toggle ensures listening history and statistics are never exposed without explicit user consent.

Learning and reference value: The codebase serves as a practical reference implementation for JWT authentication with Socket.io, MongoDB aggregation pipelines, React lazy-loading, custom audio player seek handling, and a unified CSS variable design system.

Reusable and extensible: The modular architecture (separate route/controller/context files per feature) makes it straightforward to add group chats, algorithmic recommendations, podcast support, or a CDN layer in future iterations.